Rajarata University of Sri Lanka
Department of Computing

ICT1402
PRINCIPLES OF PROGRAM DESIGN & PROGRAMMING

Lecture 01

Introduction

K.A.S.H. Kulathilake
Ph.D., M. Phil., MCS, B.Sc. (Hons) IT, SEDA(UK)

Objectives

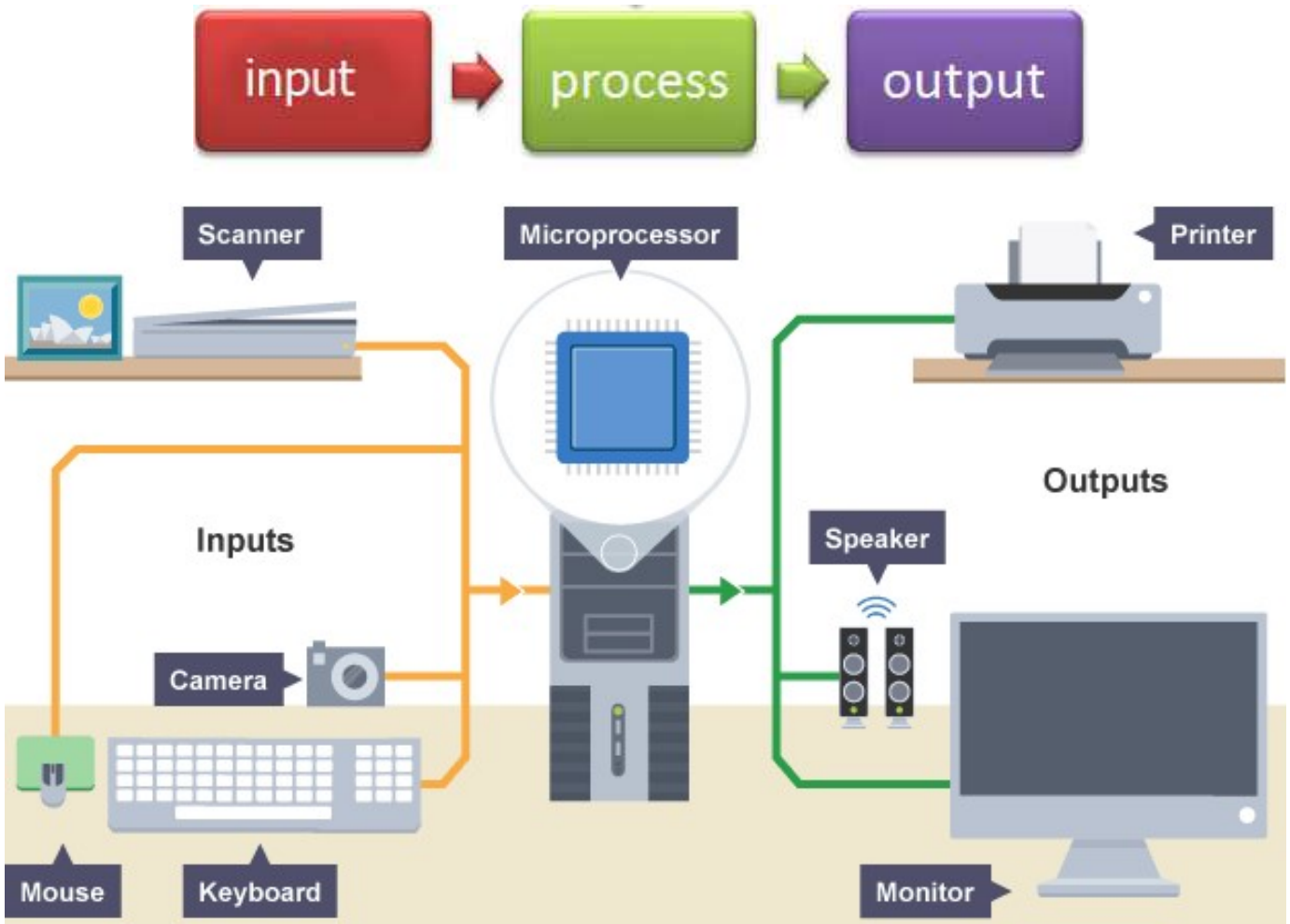
- At the end of this lecture students should be able to;
  - Define fundamentals of data processing in computers.
  - Define fundamentals of computer programming
  - Describe the aspects in the timeline of programming languages
  - Compare and contrast the generations of programming languages
  - Define the types of programming languages
  - Discuss the applications of programming languages for implementing various real-world requirements.

Computers Everywhere

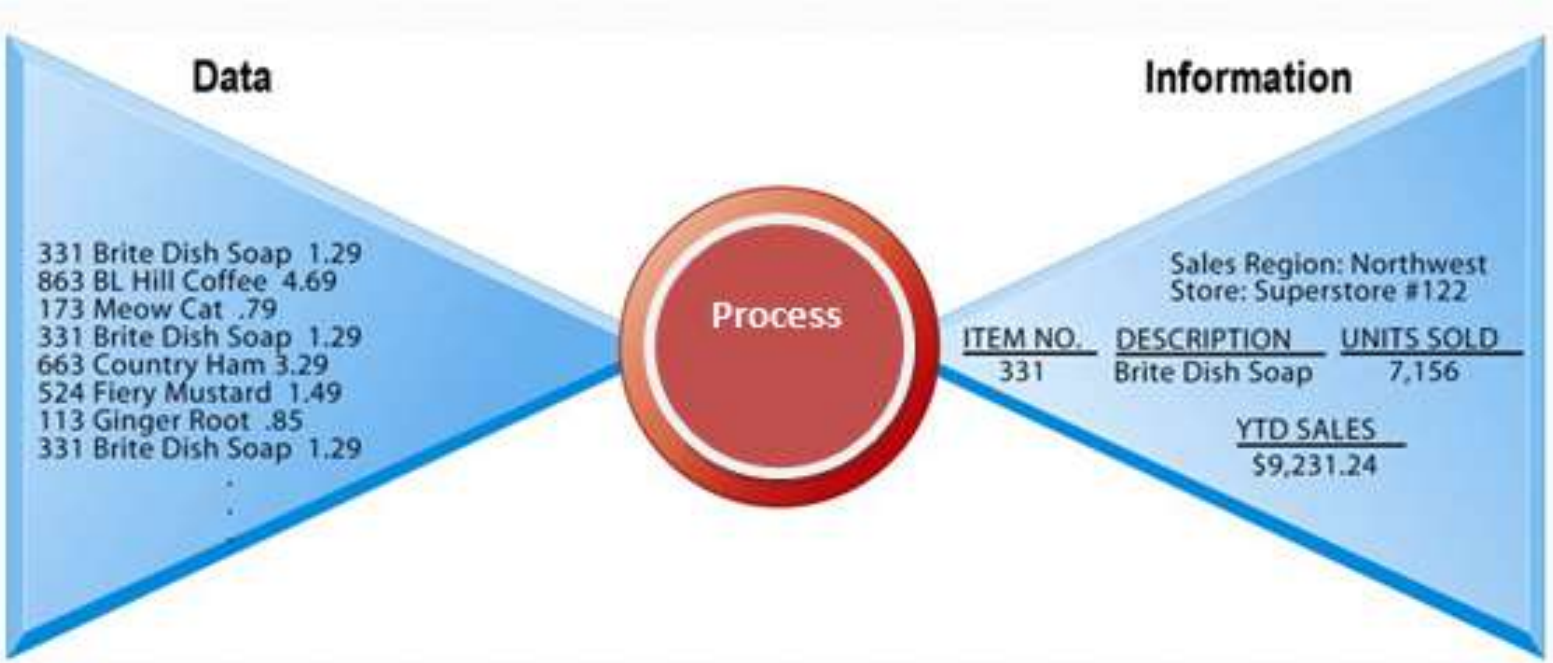
What is a Computer?

- A computer is a machine which can take instructions (data) as input, perform computations based on those instructions (process) and generate results a.k.a output (information).

What is a Computer? (Cont…)



What is a Computer? (Cont…)



- Raw data from a supermarket checkout counter can be processed and organized to produce meaningful information, such as the total unit sales of dish detergent or the total sales revenue from dish detergent for a specific store or sales territory.

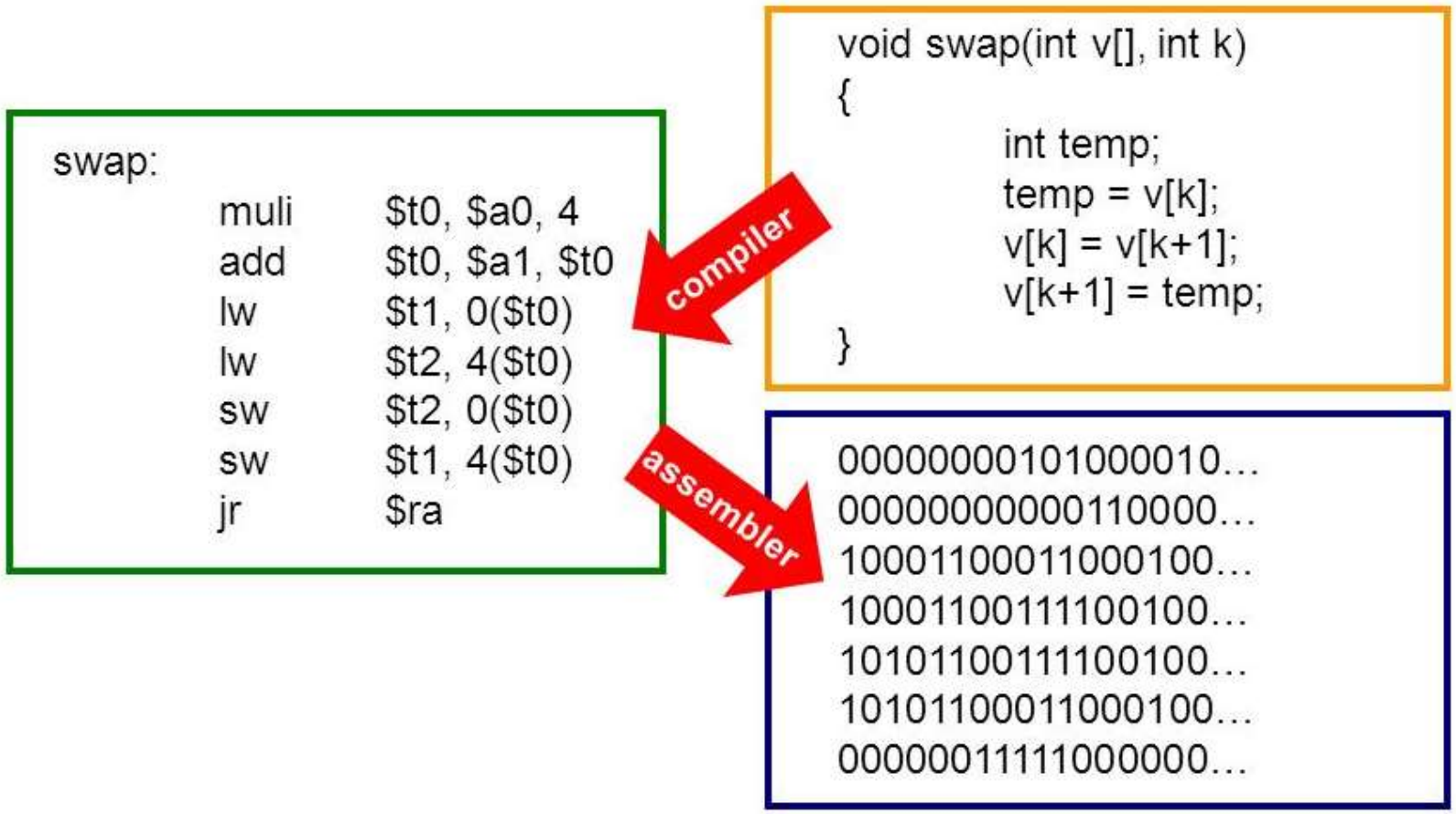
What is Processing?

- A computer uses a Central Processing Unit or CPU to do all its decision-making and data processing.
- The CPU has an internal set of instructions it follows when it receives a command (Machine Language).
- Processing data through computers is
  - Fast (Computers processing speed is measured in MIPS - Millions of Instructions Per Second.
  - Highly accurate
- The CPU follows the programmer's logic to process the data given it.

Machine Language

- **Machine Language is the only language that is directly understood by the computer.**
- It does not needs any translator program.
- The only advantage is that program of machine language run very fast.
- Each different type of CPU has its own unique machine language.

Machine Language (Cont…)



What is a Computer Program?

- A computer program is **a collection of instructions** that **performs a specific** task when executed by a computer.
- A computer requires programs to function, and typically executes the program's instructions in a central processing unit.
- A computer program is usually written by a computer programmer in a programming language.

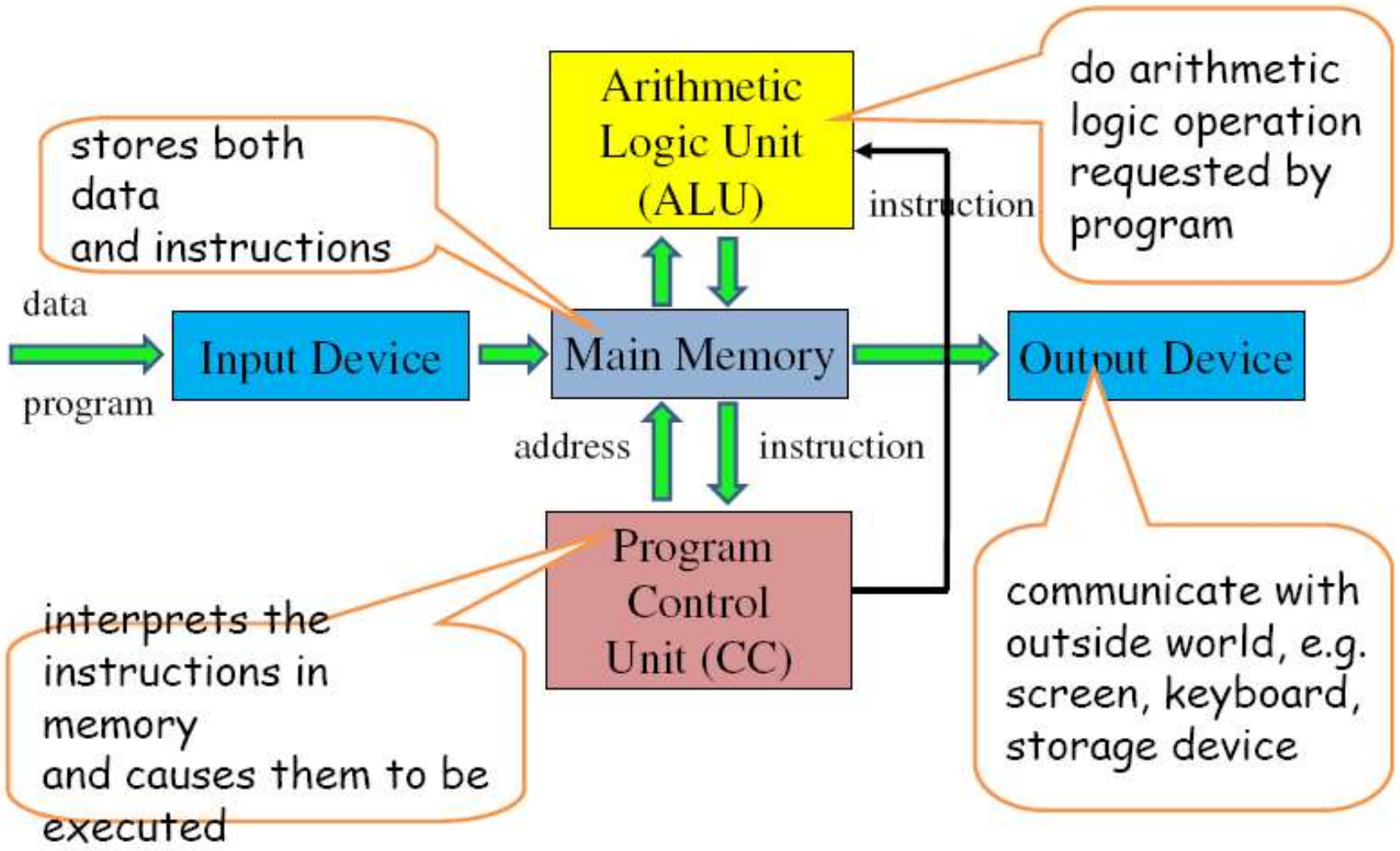
What is a Programming Language?

- **A programming language is a formal computer language designed to communicate instructions to a machine, particularly a computer.**
- Programming languages can be used to create programs to control the behavior of a machine or to express algorithms.
- A part of a computer program that performs a well-defined task is known as an **algorithm.**

Von Neumann Architecture

- Von Neumann Architecture also known as the Von Neumann model, the computer consisted of a CPU, memory and I/O devices.
- The von Neumann architecture describes a general framework, or structure, that a computer's hardware, programming, and data should follow.
- The program is stored in the memory.
- The CPU fetches an instruction from the memory at a time and executes it.

Von Neumann Architecture (Cont…)



Von Neumann Architecture (Cont…)

- Stored Program Concept
  - A program be electronically stored in binary-number format in a memory device so that instructions could be modified by the computer as determined by intermediate computational results.

From Program To Software

- A collection of computer programs, libraries and related data are referred to as **software**.
- Computer programs may be categorized along functional lines, such as application software or system software.
  - Application Software is a set of programs for a specific application.
  - System software are general programs designed for performing tasks such as controlling all operations required to move data into and out of the computer

History of Programming

Generations of Programming Languages

- There are 5 Generations
  - 1st Generation
  - 2nd Generation
  - 3rd Generation
  - 4th Generation
  - 5th Generation

Generations of Programming Languages (Cont…)

- 1st Generation
  - **Machine language** is the only programming language that the computer can understand directly without translation.
  - It is a language made up of entirely 1s and 0s.
  - There is not, however, one universal machine language because the language must be written in accordance with the special characteristics of a given processor.
  - Each type or family of processor requires its own machine language. For this reason, machine language is said to be machine-dependent (also called hardware-dependent).

Generations of Programming Languages (Cont…)

- Machine language programs have the advantage of very fast execution speeds and efficient use of primary memory.
- Use of machine language is very tedious, difficult and time consuming method of programming.
- Machine language is low-level language. Since the programmer must specify every detail of an operation, a low-level language requires that the programmer have detailed knowledge of how the computer works.
- Programmers had to know a great deal about the computer's design and how it functioned.

Generations of Programming Languages (Cont...)

- 2nd Generation
  - The first step in making software development easier and more efficient was the creation of **Assembly languages**.
  - Detailed knowledge of hardware is still required.
  - Assembly languages use mnemonic operation codes and symbolic addresses in place of 1s and 0s to represent the operation codes.
  - In this level, programmer can use abbreviation instead of having to remember lengthy binary instruction codes.
  - For example, it is much easier to remember L for Load, A for Add, B for Branch, and C for Compare than the binary equivalents i-e different combinations of 0s and 1s.

Generations of Programming Languages (Cont…)

- Only computer specialists familiar with the architecture of the computer being used can use them.
- Since the language is machine dependent, assembly languages are not easily converted to run on other types of computers.
- Before they can be used by the computer, assembly languages must be translated into machine language.
- A language translator program called an **assembler** does this conversion.

Generations of Programming Languages (Cont…)

- Assembly languages produce programs that are
  - efficient,
  - use less storage, and
  - execute much faster than programs designed using high-level languages.

```
G:\>DEBUG < HELLO.TXT
-A
1437:0100 MOV AH,9
1437:0102 MOV DX,108
1437:0105 INT 21
1437:0107 RET
1437:0108 DB 'HELLO WORLD$'
1437:0114
-R CX
CX 0000
:14
-N MYHELLO.COM
-W
Writing 00014 bytes
-Q

G:\>MYHELLO
HELLO WORLD
G:\>
```

Generations of Programming Languages (Cont…)

- 3rd Generation
  - Third generation languages, also known as **high-level languages**, are very much like everyday text and mathematical formulas in appearance.
  - They are designed to run on a number of different computers with few or no changes.
  - Most high level languages are considered to be procedure-oriented, or Procedural languages, because the program instructions comprise lists of steps, procedures, that tell the computer not only what to do but how to do it.

Generations of Programming Languages (Cont…)

- The programmer spends less time developing software with a high level language than with assembly or machine language because fewer instructions have to be created.
- **A language translator** is required to convert a high-level language program into machine language.
- Two types of language translators are used with high level languages: **compilers and interpreters**.
- e.g. FORTRAN, COBOL, BASIC, Pascal and C

Generations of Programming Languages (Cont…)

hello1.cob | sampleCOBOL.cbl

```
1      Identification Division.CRLF
2  LF
3      Program-ID. sampleCOBOL.CRLF
4  CRLF
5      Data Division.CRLF
6  CRLF
7      Procedure Division.CRLF
8      Main-Para.LF
9          Display "Hello World!"CRLF
10         Stop Run.CRLF
11
```

Generations of Programming Languages (Cont…)

- 4th Generation
  - Fourth generation languages are also known as **very high level languages**.
  - They are non-procedural languages, so named because they allow programmers and users to specify what the computer is supposed to do without having to specify how the computer is supposed to do it.
  - Consequently, fourth generation languages need approximately one tenth the number of statements that a high level languages needs to achieve the same results.
  - Because they are so much easier to use than third generation languages, fourth generation languages allow users, or non-computer professionals, to develop software.

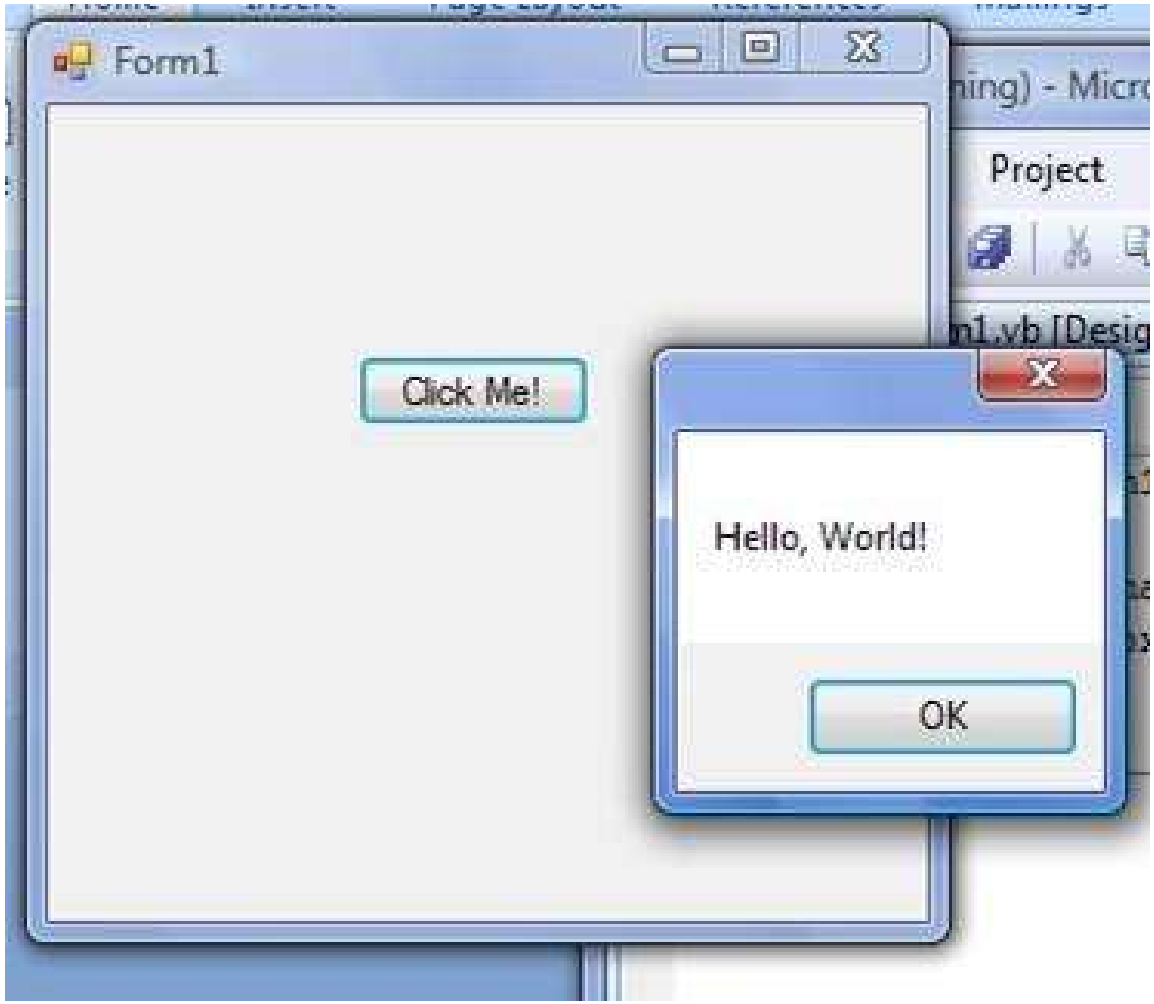
Generations of Programming Languages (Cont…)

- Objectives of fourth generation languages
  - Increasing the speed of developing programs.
  - Minimizing user effort to obtain information from computer.
  - Decreasing the skill level required of users so that they can concentrate on the application rather than the details of coding, and thus solve their own problems without the aid of a professional programmer.
  - Minimizing maintenance by reducing errors and making programs that are easy to change.
- These languages are usually used in conjunction with a database and its data dictionary.

Generations of Programming Languages (Cont...)

- Five basic types of language tools fall into the fourth generation language category.
  - Query languages
  - Report generators.
  - Applications generators.
  - Decision support systems and financial planning languages.
  - Some microcomputer application software.

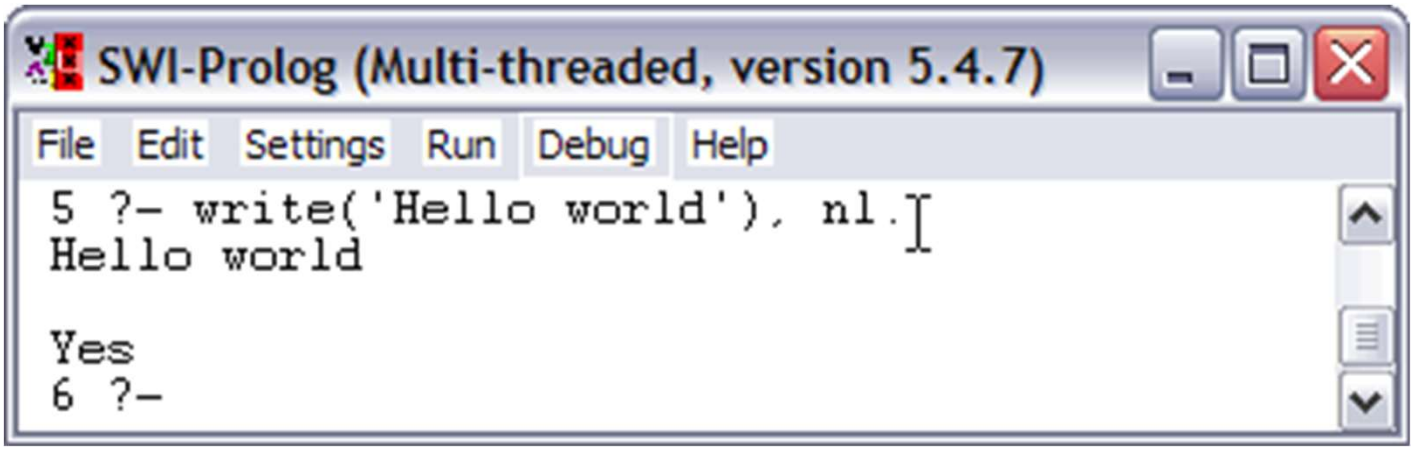
Generations of Programming Languages (Cont…)



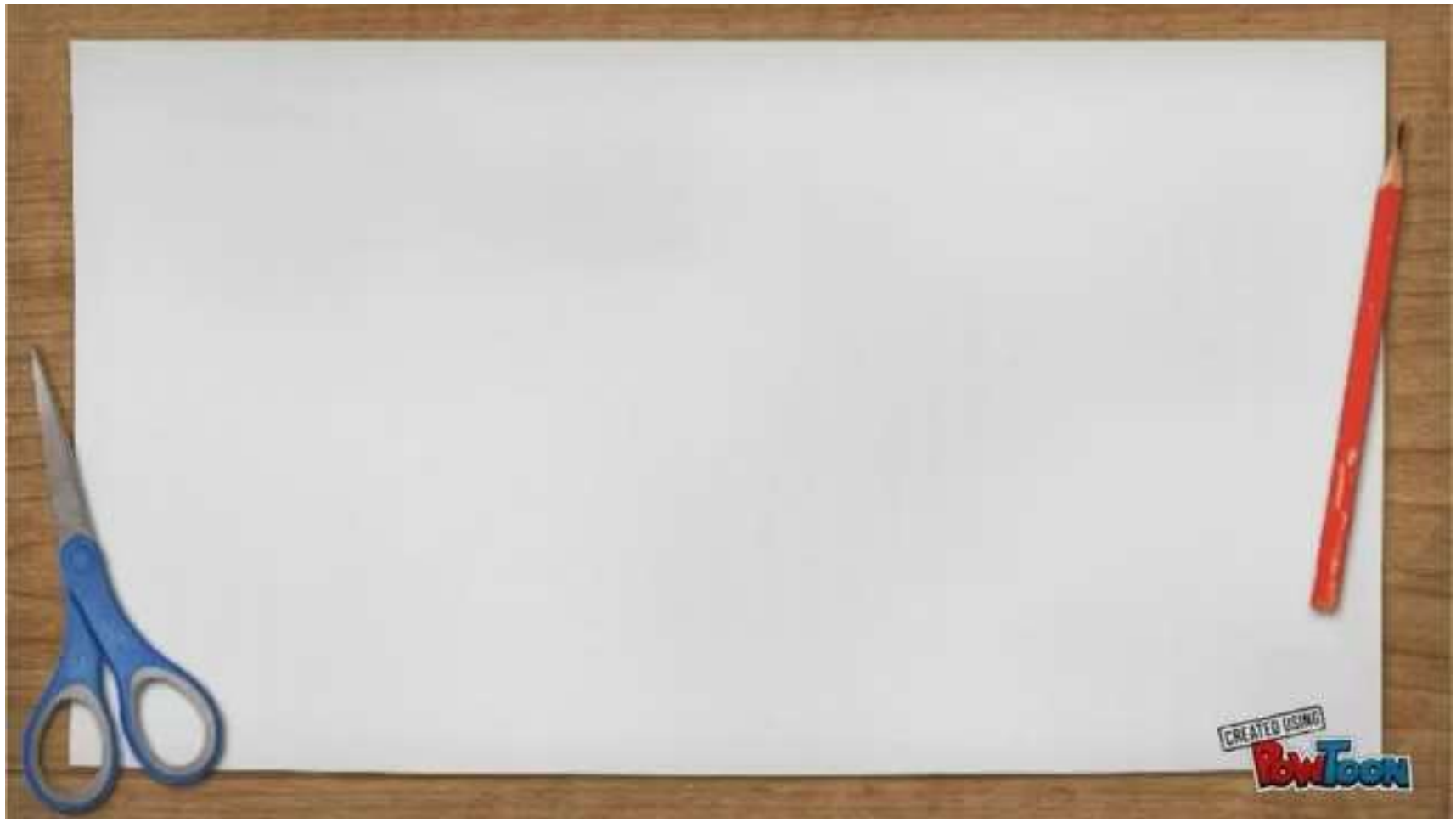
Generations of Programming Languages (Cont…)

- 5th Generation
  - The text of a natural language statement very closely resembles human speech.
  - These languages are also designed to make the computer "smarter".
  - Natural languages already available for microcomputers include Clout, Q&A, and Savvy Retriever (for use with databases) and HAL (Human Access Language).
  - The use of natural language touches on expert systems, computerized collection of the knowledge of many human experts in a given field, and artificial intelligence, independently smart computer systems.
  - e.g Prolog, OPS5 and Mercury and Lisp

Generations of Programming Languages (Cont…)



Generations of Programming Languages (Cont…)



Types of Programming Languages

- Two types of programming languages
  - Low-level programming languages
  - High-level programming languages

Generation	First	Second	Third	Fourth
Code example	10101010011000101 10011010100000010 11111111101000101	LDA 34 ADD #1 STO 34	x = x + 1	body.top { color : red; font-style : italic }
Language	(LOW) Machine Code	(LOW) Assembly Code	(HIGH) Visual Basic, C, python etc.	(HIGH) SQL, CSS, Haskell etc.
Relation to Object Code (generally)	--	one to one	one to many	one to many

Types of Programming Languages (Cont…)

- Low level programming languages
  - Provides little or no abstraction from a computer's instruction set architecture—commands or functions in the language map closely to processor instructions.
  - Generally refers to either machine code or assembly language.
  - Programs written in low-level languages tend to be relatively non-portable, mainly because of the close relationship between the language and the hardware architecture.

Types of Programming Languages (Cont…)

- Low-level languages can convert to machine code without a compiler or interpreter— (second-generation programming languages use a simpler processor called an assembler) and resulting code runs directly on the processor.
- A program written in a low-level language can be made to run very quickly, with a small memory footprint.
- Low-level languages are simple, but considered difficult to use, due to numerous technical details that the programmer must remember.
- By comparison, a high-level programming language isolates execution semantics of a computer architecture from the specification of the program, which simplifies development.

Types of Programming Languages (Cont…)

- High level programming languages
  - It is a programming language with strong abstraction from the details of the computer.
  - In comparison to low-level programming languages,
    - it may use natural language elements,
    - be easier to use,
    - or may automate (or even hide entirely) significant areas of computing systems (e.g. memory management),
    - making the process of developing a program simpler and more understandable relative to a lower-level language.

Types of Programming Languages (Cont…)

- Objectives of high-level languages
  - To relieve the programmer of the detailed and tedious task of writing programs in machine language and assembly languages.
  - To provide programs that can be used on more than one type of machine with very few changes.
  - To allow the programmer more time to focus on understanding the user's needs and designing the software required meeting those needs.

Why Many?


Microsoft
Language
Pascal
ABAP
C/AL
Cg
SYMPL
Cesil
Scala
Rust
GDL
Bistro
JOSS
Blue
Genie
Inform
SenseTalk
HAL/S
SIMSCRIPT
Revolution
Sawzall
Chapel
Rapira
CSKA
SIMPLE
ALGOL
DATATRIEVE
PL/I
ELAN
Agora
Clojure
Ajax
Not
Trac
Object
Microcode
code
AutoIt
Lynx
SAM76
Redcode
Logo
Adenine
SALSA
ATS
Fjölnir
Fuxi
Assembly
TACPOL
FP
C
shell
language
E
A+
Lisp
VB
MicroScript
Squirrel
Windows/Dos
Kaleidoscope
IMP
Fancy
SuperTalk
P
C++
ABC
LC-3
L
XOTcl
dc
Cayenne
K
Visual
Go
JavaScript
FL
JavaFX
R
Forth
b
T
Newspeak
D
Delphi
DASL
Caml
Z
GameMonkey
ECMAScript
Prolog
GJ
JASS
S
EusLisp
ObjectScript
bc
GM
Constraint
Oxygene
PEARL
G
Programming
Java
BLISS
notation
Ruby
EXEC
RPG
M
Cyclone
RPL
Script
PHP
CorVision
Action
REXX
F
Common
Agena
Little
NET
Q
Simula
Boomerang
COBOL
GraphTalk
A
ChucK
Strand
Turing
ISWIM
S/SL
PL/SQL
rexx
Arc
Moby
ORCA/Modula-2
Uniface
Python
J
Robot
SP/k
HaXe
Eiffel
Coral
WATFOR

Programming: Its context and Applications

System Software

Programming: Its context and Applications (Cont…)

Application Software

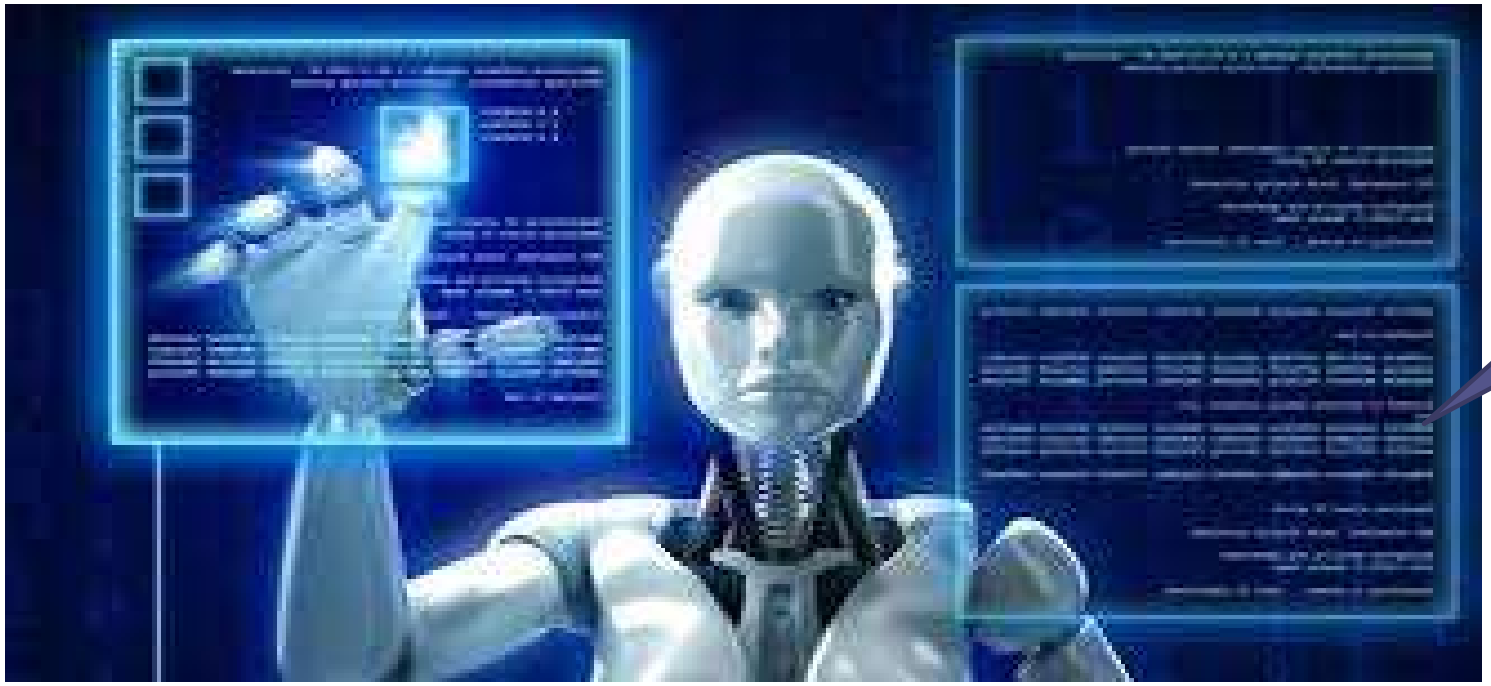
Programming: Its context and Applications (Cont…)

Mobile Apps

Programming: Its context and Applications (Cont…)

Numerous Cloud Services

Programming: Its context and Applications (Cont…)



Soft Computing
Embedded
Systems

Programming: Its context and Applications (Cont…)

Numerous User Requirements

All Industries

Objectives - Revisit

- Define fundamentals of data processing in computers.
- Define fundamentals of computer programming
- Describe the aspects in the timeline of programming languages
- Compare and contrast the generations of programming languages
- Define the types of programming languages
- Discuss the applications of programming languages for implementing various real-world requirements.

Q & A

Next: Computer Program Design